

```

%%writefile matrix_mul.cu
#include <iostream>
#include <cuda.h>
#include <cstdlib>
#include <ctime>

using namespace std;

#define TILE_SIZE 16 // 16x16 threads per block

// -----
// CUDA Kernel for Matrix Multiplication
// -----
__global__ void matrixMultiply(int *A, int *B, int *C, int N) {
    // Calculate global row and column
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < N && col < N) {
        int sum = 0;
        for (int k = 0; k < N; k++) {
            sum += A[row * N + k] * B[k * N + col];
        }
        C[row * N + col] = sum;
    }
}

// -----
// Helper Function to Print Matrices
// -----
void printMatrix(int *mat, int N, const char* name) {
    cout << "\n--- Matrix " << name << " ---\n";
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            cout << mat[i * N + j] << "\t";
        }
        cout << "\n";
    }
}

// -----
// Main Function & Benchmarking
// -----
int main() {
    int N;
    cout << "Enter the size of the square matrix (N x N): ";
    cin >> N;

    size_t size = N * N * sizeof(int);

```

```

// Allocate memory on Host
int *h_A = (int *)malloc(size);
int *h_B = (int *)malloc(size);
int *h_C = (int *)malloc(size);

// Smart Input Logic
if (N <= 5) {
    char choice;
    cout << "Do you want to enter elements manually? (y/n): ";
    cin >> choice;

    if (choice == 'y' || choice == 'Y') {
        cout << "Enter " << N * N << " elements for Matrix A (row by row):\n";
        for (int i = 0; i < N * N; i++) cin >> h_A[i];

        cout << "Enter " << N * N << " elements for Matrix B (row by row):\n";
        for (int i = 0; i < N * N; i++) cin >> h_B[i];
    } else {
        srand(time(0));
        for (int i = 0; i < N * N; i++) {
            h_A[i] = rand() % 10;
            h_B[i] = rand() % 10;
        }
    }
} else {
    // Auto-generate for massive benchmark sizes to save time
    srand(time(0));
    for (int i = 0; i < N * N; i++) {
        h_A[i] = rand() % 10;
        h_B[i] = rand() % 10;
    }
    cout << "[Auto-generating " << N << "x" << N << " matrices for benchmarking]\n";
}

// Print input matrices if they are small enough
if (N <= 5) {
    printMatrix(h_A, N, "A");
    printMatrix(h_B, N, "B");
}

// Allocate memory on Device
int *d_A, *d_B, *d_C;
cudaMalloc((void **)&d_A, size);
cudaMalloc((void **)&d_B, size);
cudaMalloc((void **)&d_C, size);

// Copy from Host to Device
cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

```

```

// Define Grid and Block dimensions (2D)
dim3 threadsPerBlock(TILE_SIZE, TILE_SIZE);
dim3 blocksPerGrid((N + TILE_SIZE - 1) / TILE_SIZE, (N + TILE_SIZE - 1) / T

// Set up CUDA Events
cudaEvent_t start, stop;
cudaEventCreate(&start);
cudaEventCreate(&stop);

// Launch Kernel and record time
cudaEventRecord(start);
matrixMultiply<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, N);
cudaEventRecord(stop);

cudaEventSynchronize(stop);

float milliseconds = 0;
cudaEventElapsedTime(&milliseconds, start, stop);

// Copy result back to Host
cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);

// Verification Output
if (N <= 5) {
    printMatrix(h_C, N, "C (Result: A * B)");
} else {
    cout << "\n[Success] " << N << "x" << N << " Matrix Multiplication comp
}

cout << "\n[TIME] GPU Execution Time: " << milliseconds << " ms\n";

// Free memory
cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
free(h_A); free(h_B); free(h_C);

return 0;
}

```

Writing matrix_mul.cu

```

!nvcc matrix_mul.cu -o matrix_mul
!./matrix_mul

```

```

nvcc warning : Support for offline compilation for architectures prior to '<comp
Enter the size of the square matrix (N x N): 3
Do you want to enter elements manually? (y/n): y
Enter 9 elements for Matrix A (row by row):
7 8 5
3 7 5
9 7 4
Enter 9 elements for Matrix B (row by row):
8 13 21

```

```
78 54 45
98 34 12
```

```
--- Matrix A ---
```

```
7      8      5
3      7      5
9      7      4
```

```
--- Matrix B ---
```

```
8      13     21
78     54     45
98     34     12
```

```
--- Matrix C (Result: A * B) ---
```

```
1170    693    567
1060    587    438
1010    631    552
```

```
[TIME] GPU Execution Time: 103.833 ms
```

```
%%writefile matrix_demo.cu
#include <iostream>
#include <cuda.h>
using namespace std;

#define N 2

__global__ void matMul(int *A, int *B, int *C) {
    int row = threadIdx.y;
    int col = threadIdx.x;
    int sum = 0;
    for (int k = 0; k < N; k++)
        sum += A[row * N + k] * B[k * N + col];
    C[row * N + col] = sum;
}

void printMatrix(int mat[N][N], const char* name) {
    cout << "Matrix " << name << ":\n";
    cout << "+-----+-----+\n";
    for (int i = 0; i < N; i++) {
        cout << "|";
        for (int j = 0; j < N; j++)
            cout << " " << mat[i][j] << "   |";
        cout << "\n";
        cout << "+-----+-----+\n";
    }
    cout << "\n";
}

int main() {
    int h_A[N][N] = {{1, 2}, {3, 4}};
    int h_B[N][N] = {{5, 6}, {7, 8}};
```

```

int h_C[N][N];

int *d_A, *d_B, *d_C;
cudaMalloc(&d_A, sizeof(h_A));
cudaMalloc(&d_B, sizeof(h_B));
cudaMalloc(&d_C, sizeof(h_C));

cudaMemcpy(d_A, h_A, sizeof(h_A), cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B, sizeof(h_B), cudaMemcpyHostToDevice);

dim3 threads(N, N);
matMul<<<1, threads>>>(d_A, d_B, d_C);

cudaMemcpy(h_C, d_C, sizeof(h_C), cudaMemcpyDeviceToHost);

cout << "==== Hardcoded Matrix Multiplication Demo =====\n\n";
printMatrix(h_A, "A");
printMatrix(h_B, "B");

cout << "Manual verification:\n";
cout << "C[0][0] = (1x5) + (2x7) = 5  + 14 = 19\n";
cout << "C[0][1] = (1x6) + (2x8) = 6  + 16 = 22\n";
cout << "C[1][0] = (3x5) + (4x7) = 15 + 28 = 43\n";
cout << "C[1][1] = (3x6) + (4x8) = 18 + 32 = 50\n\n";

printMatrix(h_C, "C = A x B (GPU Result)");

cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
return 0;
}

```

Overwriting matrix_demo.cu

```

!nvcc matrix_demo.cu -o matrix_demo
!./matrix_demo

```

```

nvcc warning : Support for offline compilation for architectures prior to '<comp
==== Hardcoded Matrix Multiplication Demo =====

```

Matrix A:

```

+-----+-----+
|  1    |  2    |
+-----+-----+
|  3    |  4    |
+-----+-----+

```

Matrix B:

```

+-----+-----+
|  5    |  6    |
+-----+-----+
|  7    |  8    |
+-----+-----+

```

Manual verification:

$$C[0][0] = (1 \times 5) + (2 \times 7) = 5 + 14 = 19$$

$$C[0][1] = (1 \times 6) + (2 \times 8) = 6 + 16 = 22$$

$$C[1][0] = (3 \times 5) + (4 \times 7) = 15 + 28 = 43$$

$$C[1][1] = (3 \times 6) + (4 \times 8) = 18 + 32 = 50$$

Matrix C = A x B (GPU Result):

```
+-----+-----+
|  19   |  22   |
+-----+-----+
|  43   |  50   |
+-----+-----+
```

Start coding or [generate](#) with AI.